MNXB11 Final Project Report

Team 3:

Rebecca Batchelor,

Zuzanna Popek,

Jie Qi,

Oscar Tang

November 12, 2026

Contents

1 Introduction **1**

1.1 Team organisation and version control 1

2 Data handling **2**

2.1 Data cleaning 2

3 File Processing **2**

4 Data analysis **3**

4.1 Falsterbo's seasonal temperature cycle 3

4.2 Temperature distribution on 30 June in Falsterbo 4

4.3 Annual cold days in Lund 5

5 Conclusion **7**

Use of AI tools **7**

References **7**

1 Introduction

A temperature archive can answer different questions depending on how the observations are grouped. Grouping by calendar day reveals the seasonal cycle. Keeping just one calendar day shows how much its temperature varies from year to year. Counting days below a fixed threshold gives a measure that can be compared across years. We apply these three approaches to historical air-temperature observations from the Swedish Meteorological and Hydrological Institute (SMHI) [**smhi**, **instructions**]. The first two analyses use Falsterbo and follow Tasks 3.2 and 3.1 of the course instructions. The annual count of days below freezing in Lund is our original analysis.

The course archive includes station-specific CSV files with metadata, observation dates and UTC times, air temperatures, and quality markers. We retain the original files, write cleaned copies and deletion logs, and convert the retained measurements into a shared ROOT tree [**root**, **repo**]. Each analysis then selects the station and dates it needs from the same data structure (Figure 1).

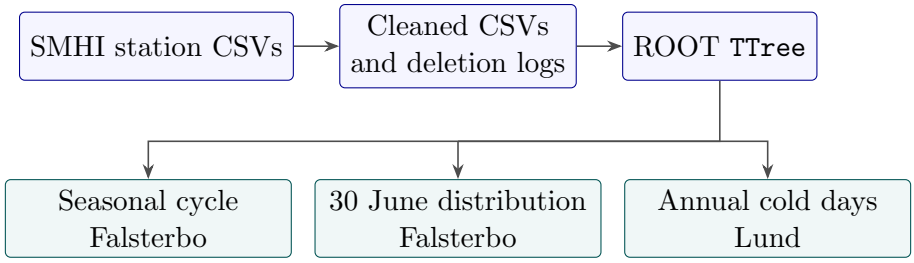


Figure 1: Project workflow from the source files to three analyses. The cleaned CSVs feed the ROOT conversion. The deletion logs preserve the evidence for the cleaning decisions.

1.1 Team organisation and version control

Jie Qi developed the cleaning tools and the Lund cold-day analysis. Oscar Tang developed the CSV-to-ROOT conversion and its batch script. Zuzanna Popek developed the calendar-day climatology, and Rebecca Batchelor developed the selected-day distribution. The team integrated the three analyses into a shared executable and recorded responsibilities in the repository work plan [**repo**].

We used a shared repository with team members as collaborators, corresponding to Policy B in the course instructions. Working on development branches allowed members to edit independent modules in parallel and integrate through GitHub pull requests. Direct collaborator access made it possible to share and test changes without requiring the release manager to push each contribution on behalf of its author.

2 Data handling

2.1 Data cleaning

The cleaner processes the 11 standard station CSVs in the course archive. The files combine measurement records with metadata and use semicolon-separated fields. The C++17 batch cleaner checks measurement rows in the order in which SMHI supplied them. Table 1 shows the four criteria. A row is removed if it fails one or more criteria.

Check	Criterion for flagging a measurement row
Field format	The date, UTC time or numerical temperature cannot be parsed in the expected format.
Timestamp order	The timestamp repeats or precedes that of the preceding raw row, when both timestamps are valid.
Temperature-change rate	The absolute change exceeds 3 °C per hour, using the elapsed time between consecutive raw rows with valid temperatures and a positive time difference.
Temperature range	The recorded temperature is outside the inclusive interval $[-45, 35]$ °C.

Table 1: Coarse-cleaning rules used by the batch cleaner. The rate threshold uses the actual observation interval in hours.

Both pairwise checks refer to adjacent rows in the original sequence, including cases where the preceding row has itself been flagged. This keeps the decision for each pair traceable to the source file. The cleaned copies retain the metadata and unflagged rows. A separate CSV per station records the original line number, date, UTC time, temperature, applicable error codes and complete original row. The optional Python tool exposes the same checks and plots the raw temperature, sampling interval, temperature difference and rate of change for interactive inspection.

The thresholds are project choices for coarse screening. Rapid genuine changes can exceed 3 °C per hour, and a bad measurement can cause the following valid measurement to fail the pairwise rate check. Sampling schedules also vary over time. A long interval can reflect either a change in sampling or missing observations. The pipeline leaves such gaps in place, so daily averages are averages of available readings and can be sensitive to the times sampled.

3 File Processing

To facilitate the data analysis it was decided to store the and format the data into a tree which would allow for a more direct interface with the ROOT framework. This means that the program has to find the correct line of the input file and be able to split each line into individual substrings in each line to then make an object and fill the tree.

This implementation achieves this by iterating through the document using a while loop and checking each line for a quality flag. This works since only the lines containing data have this flag and also comes with the additional benefit of being able to select which quality flags the program accepts and stores in the tree.

The tree store the data in the *ClimateData* class which is a custom class derived from *TObject*. The class consists of member function that store each individual unit of time that occurs in the raw data file, the measurement, and the station name. The class has getter for every member variable and has four other getter to get formatted timestamps and dates. These special getters each time unit being a member variable and the station name are meant to allow for greater sorting and comparing of the data set for the data analysis.

4 Data analysis

4.1 Falsterbo's seasonal temperature cycle

Task 3.2 asks how the typical daily temperature and its variability change over the calendar year. In `CalculateDailyMean`, ROOT selects Falsterbo observations and the program accumulates temperature sums and counts in arrays indexed by year and calendar day. A 365-day calendar omits 29 February so that a given index refers to the same date in every year.

For year y and calendar day d, let $n_{y,d}$ be the number of available readings and $T_{y,d,i}$ their temperatures. The program first computes a mean for each observed date and then averages those means across the years Y_d with observations on that date:

$$\bar{T}_{y,d} = \frac{1}{n_{y,d}} \sum_{i=1}^{n_{y,d}} T_{y,d,i}, \qquad \mu_d = \frac{1}{|Y_d|} \sum_{y \in Y_d} \bar{T}_{y,d}.$$

This gives each observed year equal weight for a given date, even when some years have more measurements per day. The spread of the yearly means is calculated with ROOT's `TMath::RMS`. The `PlotDailyMean` function stores the mean and spread in the content and error of each `TH1D` bin and draws them on a ROOT canvas.

Figure 2 shows a clear seasonal cycle. Average temperatures rise from winter through spring, peak in summer and fall through autumn. The spread is generally larger in winter, so the difference between two winters can be substantial even on the same calendar date. The bars measure historical variability. An uncertainty on μ_d would require accounting for the number of contributing years and any dependence between observations.

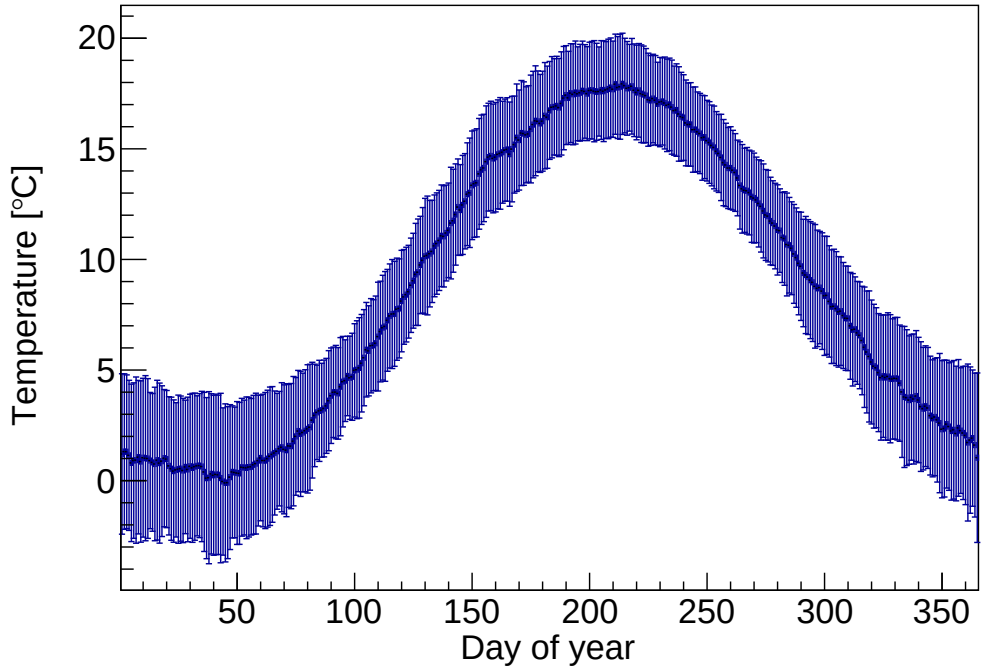


Figure 2: Mean daily temperature in Falsterbo across the calendar year. Each point combines the available yearly daily means for one date. The bars show the year-to-year standard deviation for that date. Day 1 is 1 January.

4.2 Temperature distribution on 30 June in Falsterbo

Task 3.1 asks what range of temperatures is typical on a particular date. Rebecca's analysis uses 30 June in Falsterbo. `dailyMeansPerYear` uses a ROOT `TEntryList` to select the station, month and day and groups the retained observations by year. The current default interval is 1880–2025. Each year with observations contributes one mean temperature, so a year with more readings on 30 June contributes the same weight as a year with fewer.

The histogram in Figure 3 shows that the temperature on 30 June varies appreciably across years, with most entries concentrated near the centre and fewer toward the tails. This is the distribution underlying one calendar date in the seasonal analysis. The current `DayHistogram` implementation fills a `TH1D` with 100 bins over 5 to 30 °C. It uses the histogram mean $\hat{\mu}$ and standard deviation $\hat{\sigma}$ to parameterise a Gaussian overlay. The expected curve height scales with the number of observations and the bin width, so it is expressed in the same units as the histogram.

The program also compares two estimates for the probability of a mean temperature between 16 and 18 °C. The empirical estimate divides the number of yearly daily means in $[16, 18)$ °C by the number available. The Gaussian estimate integrates the model over

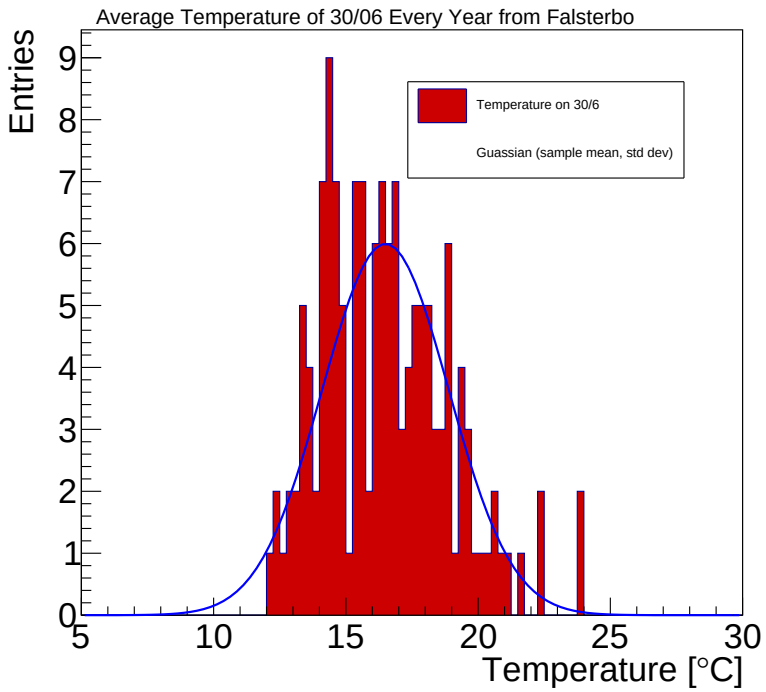


Figure 3: Archived Falsterbo histogram for 30 June with a Gaussian overlay. The entries are daily means from individual years. The curve summarizes the centre and spread of the observed distribution.

the same interval:

$$\hat{p}_{\mathrm{emp}} = \frac{\#\{y : 16 \le \bar{T}_{y,d} < 18\}}{|Y_d|}, \qquad \hat{p}_{\mathrm{G}} = \Phi\left(\frac{18-\hat{\mu}}{\hat{\sigma}}\right) - \Phi\left(\frac{16-\hat{\mu}}{\hat{\sigma}}\right).$$

Here Φ is the standard normal cumulative distribution. The empirical estimate describes the recorded sample. The Gaussian estimate smooths the distribution and also supplies a model-based upper-tail probability for $T > 20\,^{\circ}\mathrm{C}$. Both pool the available years, so interpreting them as forecasts would require assessing whether the distribution is stable over time.

4.3 Annual cold days in Lund

We chose Lund for its long local record and asked whether days with a mean temperature below $0\,^{\circ}\mathrm{C}$ have become less frequent. `PlotAnnualColdDays` reads the Lund entries with `TTreeReader` and groups the chronological observations by date. The code rounds each reading to tenths of a degree and sums the resulting integers so that opposite readings at that precision cancel exactly. The sum and mean have the same sign, so a date counts once if its sum is negative. An exact zero is excluded.

For each year y, the count is

$$C_y = \sum_{d \in D_y} \mathbf{1}\{\bar{T}_{y,d} < 0\,^\circ\mathrm{C}\},$$

where D_y is the set of dates with at least one retained observation in that year. The program plots one point per observed year and fits a straight line with ROOT's `TGraph::Fit("pol1")`. Multiplying the slope per year by ten expresses the change in days per decade.

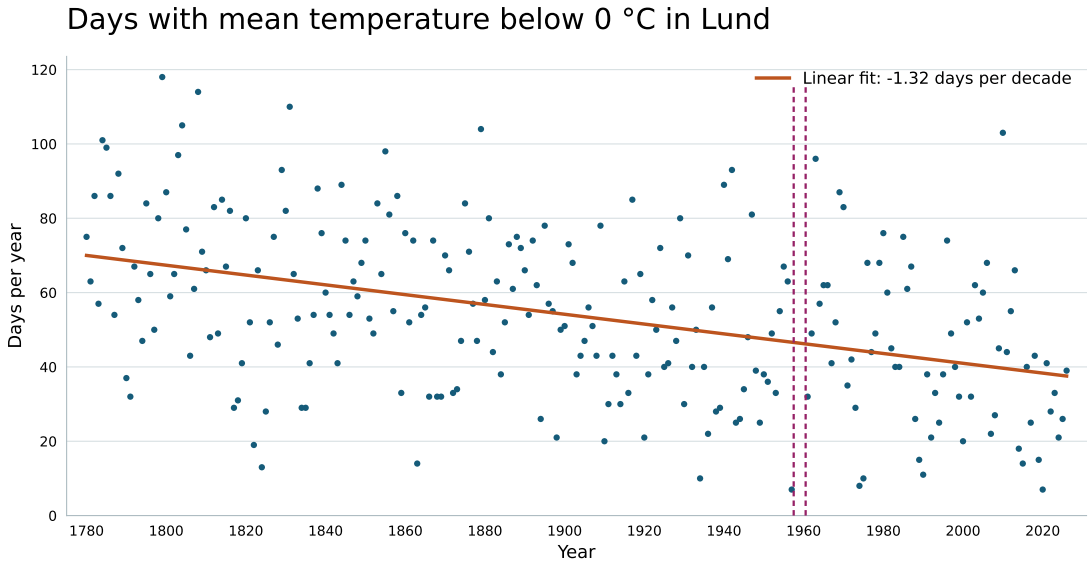


Figure 4: Annual cold days in Lund and the least-squares trend, plotted from the archived yearly counts. Each point represents one observed year. Two dashed vertical lines bracket the missing years 1958–1960. Years with partial coverage remain in the series.

The reference series contains 244 annual counts spanning 1780–2026. Its fitted slope is −1.32 cold days per decade and the linear fit accounts for about 17% of the variation in annual counts ($R^2 = 0.169$, calculated from the archived CSV). The count reached 118 in 1799 and fell to 7 in both 1957 and 2020. Counts still vary markedly in recent decades, including 103 cold days in 2010. The line in Figure 4 therefore summarises a long-term decrease alongside substantial year-to-year variation.

During the project presentation, the instructor asked why the missing years near 1960 appeared to have points on the scatter plot. We checked the archived annual counts and found that the series jumps from 1957 to 1961. Figure 4 marks the gap with two dashed vertical lines. The gap covers only three years on an axis spanning nearly 250 years, and the fitted line continues through it. Both features make the missing years easy to overlook.

For partially observed years, missing days can lower the count, especially if the missing period falls in a cold season. As a check on the final year, we recalculated the fit from the archived counts through 2025 and obtained −1.32 days per decade to two decimal

places. This tests the influence of the endpoint only. A broader sensitivity test would compare seasons or years with similar observational coverage and sampling schedules. The fitted slope describes the available Lund record.

5 Conclusion

The shared SMHI-to-ROOT workflow produced three views of the temperature record. Falsterbo's calendar-day averages show the seasonal cycle and the variability around it. The 30 June histogram resolves that variability into a distribution for a specific date and illustrates how empirical and model-based probabilities can be computed from the same yearly means. In Lund, the annual count of days with mean temperature below freezing decreases by approximately 1.32 days per decade in the archived reference series.

The principal limitation is uneven observation coverage. Changes in measurement schedules and gaps affect the daily means and can reduce annual cold-day counts. The cleaning logs and archived figures make the data selection and resulting summaries traceable. The largest remaining improvement for the Lund result would be to compare the trend across years with similar coverage during the cold months.

Use of AI tools

AI assistance was used during development to debug the calendar-day analysis, translate familiar Python operations into C++ and diagnose compilation issues. It also supported the preparation and integration of the cleaning and cold-day modules. For this report, AI assisted in organising and drafting the LaTeX, checking descriptions against the source code and recomputing the Lund fit diagnostics from the archived annual counts. These uses supported implementation and documentation. The data, plots and analysis methods are traceable to the project repository.

References

[1] Swedish Meteorological and Hydrological Institute (SMHI). *Air temperature observations and long temperature series.* `https://www.smhi.se/data/temperatur-och-vind/temperatur/airtemperatureInstant`.

[2] Lund University. *MNXB11 Project Instructions.* Course handout, Sections 1–3 and Annex B.

[3] CERN. *ROOT Data Analysis Framework.* `https://root.cern/`.

[4] MNXB11 Team 3. *Final project repository*. https://github.com/0scarlight/MNXB11-team_3-final_Project.